CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 1 Analysis of Algorithms

Midterm summary · Goodrich Chapter 1

Muharib

What's inside

01 The idea

02 Running time

03 Experimental or theoretical?

04 Pseudocode and the RAM model

05 Primitive operations

06 Counting the operations of arrayMax

07 Estimating time and the growth rate

08 Big-Oh

09 Big-Oh rules

10 Prefix averages

11 Logarithm and exponent rules

12 Relatives of Big-Oh

13 Review sheet

01 The idea

The lecture answers one question: how do we measure the speed of an algorithm without running it on a machine? We count the simple operations it performs, write the count as a function of the input size `n`, and then simplify it with Big-Oh.

> An algorithm is a step-by-step procedure for solving a problem in a finite amount of time.

What the midterm takes from it

A fixed question: count the primitive operations of **arrayMax**. Two true/false statements about theoretical and experimental analysis are also repeated word for word.

02 Running time

- The running time grows with the input size.
- The average case is often difficult to determine.
- We focus on the worst case. It is easier to analyze, and it is crucial to applications such as games, finance and robotics.

Case	Meaning	Lecture example
Best case	The fewest operations.	Insertion sort on numbers that are already sorted: `O(n)`.
Average case	The average over all inputs.	Difficult to determine.
Worst case	The most operations. This is the upper bound.	Insertion sort on reverse-sorted numbers: `O(n²)`. Linear search when the element is not present.

03 Experimental or theoretical?

1 Experimental Studies

1. Write a program implementing the algorithm.
2. Run the program with inputs of varying size and composition.
3. Use a method like System.currentTimeMillis() to measure the actual running time.
4. Plot the results.

2 Theoretical Analysis

1. Uses a high-level description of the algorithm instead of an implementation.
2. Characterizes running time as a function of the input size, n.
3. Takes into account all possible inputs.
4. Evaluates the speed of an algorithm independent of the hardware/software environment.

The experimental approach has three limitations:

1. It is necessary to implement the algorithm, which may be difficult.
2. Results may not be indicative of the running time on other inputs not included in the experiment.
3. In order to compare two algorithms, the same hardware and software environments must be used **and the same inputs**.

Two statements that appeared in the midterm

> In order to compare two algorithms, the same hardware and software environments must be used **without any consideration of inputs type and its size**.

False. The same inputs must be used as well.

> Theoretical analysis allows us to evaluate the speed of an algorithm independent of the hardware/software environment.

True. This is the fourth property of theoretical analysis.

04 Pseudocode and the RAM model

Pseudocode is a high-level description of an algorithm. It is more structured than English prose, less detailed than a program, and it hides program design issues.

```
Algorithm arrayMax(A, n)
    Input   array A of n integers
    Output  maximum element of A
    currentMax ← A[0]
    for i ← 1 to n − 1 do
        if A[i] > currentMax then
            currentMax ← A[i]
    return currentMax
```

The RAM model (Random Access Machine) is the imaginary machine we analyze on:

- It has one CPU.
- It has an unbounded bank of numbered memory cells, each holding a number or a character.
- Accessing any cell takes unit time.

05 Primitive operations

They are the basic computations an algorithm performs. They are identifiable in pseudocode, largely independent of the programming language, and each is assumed to take a constant amount of time in the RAM model.

Operation	Example	Count
Evaluating an expression	`i + 1 , A[i] > currentMax`	1
Assigning a value to a variable	`currentMax ← …`	1
Indexing into an array	`A[i]`	1
Calling a method	`arrayMax(A, n)`	1
Returning from a method	`return currentMax`	1

06 Counting the operations of arrayMax

This is the midterm question. Learn the table and where each number comes from:

Line	# operations	Why
`currentMax ← A[0]`	`2`	Indexing and assignment.
`for i ← 1 to n − 1 do`	`1 + n`	`i ← 1` runs once. The test `i ≤ n − 1` runs `n` times, and the last one fails and ends the loop.
`if A[i] > currentMax then`	`2(n − 1)`	Indexing and a comparison, in every iteration.
`currentMax ← A[i]`	`2(n − 1)`	Indexing and assignment. In the worst case it runs in every iteration.
`{ increment counter i }`	`2(n − 1)`	An addition and an assignment.
`return currentMax`	`1`	The return.
Total	**`7n − 2`**	The loop runs `n − 1` times.

```
Worst case:   2 + (1 + n) + 2(n − 1) + 2(n − 1) + 2(n − 1) + 1  =  7n − 2
```

```
Best case:    2 + (1 + n) + 2(n − 1) +     0     + 2(n − 1) + 1  =  5n
```

The worst case is an array sorted in increasing order, so every element beats the current maximum. The best case is when `A[0]` is the maximum, so the fourth line never runs.

The midterm's version

In the 451 and 442 midterms the instructor added the line `Temp ← A[0]` after the first line. That adds two operations, so the answer becomes `7n − 2 + 2 = 7n`.

Note: slide 23 says "at most 7n − 1", while slides 12 and 14 say `7n − 2`. The table gives `7n − 2`.

07 Estimating time and the growth rate

```
a = time taken by the fastest primitive operation
```

```
b = time taken by the slowest primitive operation
```

```
a(7n − 2)  ≤  T(n)  ≤  b(7n − 2)
```

So `T(n)` is bounded by two linear functions, which makes it linear.

- Changing the hardware or software affects `T(n)` by a constant factor only.
- It does not change the growth rate.
- The growth rate is not affected by constant factors or lower-order terms.

Function	Type
$10^2n + 10^5$	Linear
$10^5n^2 + 10^8n$	Quadratic

Functions ordered from the slowest growth to the fastest:

```
1  <  log n  <  n  <  n log n  <  n²  <  n³  <  2ⁿ
```

08 Big-Oh

Given functions f(n) and g(n), we say that f(n) is O(g(n)) if there are positive constants c and n_0 such that

$f(n) \leq c \cdot g(n)$ for $n \geq n_0$

In words: `g(n)` times a constant stays above `f(n)` from some point n_0 onward. Big-Oh gives an upper bound.

Ex **Show that `2n + 10` is `O(n)`**

$$2n + 10 \leq cn$$

$$(c - 2)n \geq 10$$

$$n \geq \frac{10}{c - 2}$$

Pick $c = 3$ and $n_0 = 10$

Ex **Show that n^2 is not `O(n)`**

$$n^2 \leq cn$$

$$n \leq c$$

This cannot hold, because `c` must be a constant while `n` grows without bound.

f(n)	Big-Oh	c	n_0
$7n - 2$	$O(n)$	7	1
$3n^3 + 20n^2 + 5$	$O(n^3)$	4	21
$3 \log n + \log \log n$	$O(\log n)$	4	2

A quick way to find c and n_0

Raise every term to the highest power. For example $3n^3 + 20n^2 + 5 \leq 3n^3 + 20n^3 + 5n^3 = 28n^3$ for all $n \geq 1$, so $c = 28$ and $n_0 = 1$. Any pair that satisfies the inequality is correct. There is no single answer.

09 Big-Oh rules

- If $f(n)$ is a polynomial of degree d, it is $O(n^d)$. Drop the lower-order terms and the constant factors.
- Use the smallest possible class: say $2n$ is $O(n)$, not $O(n^2)$.
- Use the simplest expression: say $3n + 5$ is $O(n)$, not $O(3n)$.

	f(n) is O(g(n))	g(n) is O(f(n))
g(n) grows more	Yes	No
f(n) grows more	No	Yes
Same growth	Yes	Yes

Asymptotic analysis has two steps. We find the worst-case number of primitive operations as a function of n, then we express it in Big-Oh. For example **arrayMax** executes $7n - 2$ operations, so we say it "runs in $O(n)$ time".

10 Prefix averages

We want an array `A` where `A[i]` is the average of the first `i + 1` elements of `X`:

$$A[i] = \frac{X[0] + X[1] + \ldots + X[i]}{i + 1}$$

1 Quadratic · O(n²)

```
prefixAverages1(X, n)                  # ops
A ← new array of n integers             n
for i ← 0 to n − 1 do                   n
    s ← X[0]                            n
    for j ← 1 to i do                   1+2+…+(n−1)
        s ← s + X[j]                    1+2+…+(n−1)
    A[i] ← s / (i + 1)                  n
return A                                1
```

2 Linear · O(n)

```
prefixAverages2(X, n)                  # ops
A ← new array of n integers             n
s ← 0                                   1
for i ← 0 to n − 1 do                   n
    s ← s + X[i]                        n
    A[i] ← s / (i + 1)                  n
return A                                1
```

The first one restarts the sum every time, so the inner loop runs `1 + 2 + … + (n − 1)` times. The second keeps a running sum `s`, so one loop is enough.

$$\text{Arithmetic progression:} \quad 1 + 2 + \dots + n = \frac{n(n+1)}{2} \quad \to \quad O(n^2)$$

11 Logarithm and exponent rules

$$\log_b(xy) = \log_b x + \log_b y$$

$$\log_b\left(\frac{x}{y}\right) = \log_b x - \log_b y$$

$$\log_b(x^a) = a \cdot \log_b x$$

$$\log_b a = \frac{\log_x a}{\log_x b}$$

$$a^{(b+c)} = a^b \cdot a^c$$

$$a^{(bc)} = (a^b)^c$$

$$\frac{a^b}{a^c} = a^{(b-c)}$$

$$b = a^{(\log_a b)}$$

$$b^c = a^{(c \cdot \log_a b)}$$

12 Relatives of Big-Oh

Notation	Condition	Meaning
Big-Oh O	$f(n) \leq c \cdot g(n)$	`f` is asymptotically less than or equal to `g` (an upper bound).
Big-Omega Ω	$f(n) \geq c \cdot g(n)$	`f` is asymptotically greater than or equal to `g` (a lower bound).
Big-Theta Θ	$c' \cdot g(n) \leq f(n) \leq c'' \cdot g(n)$	`f` is asymptotically equal to `g`.
little-oh o	$f(n) \leq c \cdot g(n)$ for any $c > 0$	`f` is asymptotically strictly less than `g`.
little-omega ω	$f(n) \geq c \cdot g(n)$ for any $c > 0$	`f` is asymptotically strictly greater than `g`.

The difference between big and little: for O and Ω one constant `c` is enough. For o and ω the condition must hold for any `c > 0`.

13 Review sheet

Question	Answer
Which case do we analyze?	The worst case.
Operations of **arrayMax**?	`7n - 2` in the worst case and `5n` in the best. With the Temp line: `7n`.
Definition of Big-Oh?	$f(n) \leq c \cdot g(n)$ for $n \geq n_0$
What does new hardware change?	Only the constant factor, not the growth rate.
prefixAverages1	$O(n^2)$
prefixAverages2	$O(n)$
$1 + 2 + \ldots + n$	$\frac{n(n+1)}{2}$